GUIDE COMPLET

Les Hooks de React

Comprendre leur rôle, savoir lequel choisir et les utiliser sans pièges.

Exemples pratiques · React 19 · formulaires · effets · performance · Hooks personnalisés

Un document de référence pour apprendre les Hooks pas à pas et les appliquer dans une application React comme GAIN OFFICE.

Version préparée en octobre 2026 · Les exemples sont écrits en JavaScript et JSX.

Sommaire

Sommaire 2
1. Comprendre les Hooks 3
2. Hooks d’état 4
3. Partager une valeur avec useContext 6
4. Hooks de références 7
5. Synchroniser avec l’extérieur : les Effects 8
6. useEffectEvent - logique non réactive liée à un effet 10
7. Rendu et performance 11
8. Identifiants et systèmes externes 13
9. Actions, formulaires et mises à jour optimistes 15
10. Créer un Hook personnalisé 17
11. Quel Hook choisir ? 18
12. Erreurs fréquentes et bonnes pratiques 19
13. Fiche de référence des Hooks 20
14. Glossaire 21

1. Comprendre les Hooks

Un Hook est une fonction spéciale de React qui donne à un composant fonctionnel accès à une capacité React : garder un état, lire un contexte, gérer une référence, se synchroniser avec le navigateur ou optimiser un rendu.

Les Hooks permettent aussi de partager une logique entre composants. Ils remplacent souvent les classes pour gérer l'état et le cycle de vie, mais ils ne sont pas une simple collection d'outils à ajouter partout.

> Idée centrale Un Hook doit être appelé pendant le rendu d'un composant React ou d'un autre Hook personnalisé. React suit leur ordre d'appel pour associer correctement chaque état au bon composant.

Le rendu en trois étapes

Étape	Ce qui se passe
1. Rendu	React appelle le composant. Les Hooks lisent l'état et calculent le JSX à afficher.
2. Affichage	React met à jour le DOM du navigateur pour refléter le JSX.
3. Effets	Après l'affichage, les effets synchronisent le composant avec les systèmes extérieurs, si nécessaire.

Les règles obligatoires

- Appeler les Hooks uniquement au niveau supérieur d'un composant ou d'un Hook personnalisé.
- Ne pas les appeler dans un `if`, une boucle, une fonction imbriquée, un gestionnaire de clic ou un bloc `try/catch`.
- Ne pas appeler un composant comme une fonction ordinaire. Écrire `<MaPage />`, pas `MaPage()`.
- Un Hook personnalisé commence par `use` et n'est appelé que par un composant React ou un autre Hook.

```
function Compteur() {
  const [nombre, setNombre] = useState(0);

  if (nombre > 5) {
    // Ne pas appeler un Hook ici.
  }

  return <button onClick={() => setNombre(n => n + 1)}>{nombre}</button>;
}
```

> Exception à connaître L'API `use` de React porte un nom proche d'un Hook, mais la documentation React la classe comme une API. Elle peut notamment lire un contexte ou une promesse et possède des règles particulières.

2. Hooks d’état

L’état est une donnée dont la modification doit provoquer un nouvel affichage : le texte d’un champ, une liste chargée, un filtre sélectionné, une modale ouverte ou fermée.

2.1 useState - état simple

`useState` crée une valeur d’état et une fonction qui demande à React de la remplacer. React réaffiche le composant avec la nouvelle valeur.

Quand l’utiliser : La plupart des états locaux : saisie, chargement, erreurs, sélection, visibilité d’une modale.

Forme générale : `const [valeur, setValeur] = useState(valeurInitiale)`

```
import { useState } from "react";

function Recherche() {
  const [terme, setTerme] = useState("");

  return (
    <input value={terme} onChange={e => setTerme(e.target.value)} />
  );
}
```

À retenir Pour calculer la prochaine valeur depuis l’ancienne, utiliser une fonction comme `setNombre(n => n + 1)`. Ne pas modifier directement un tableau ou un objet déjà stocké dans l’état.

Mettre à jour objets et tableaux sans les muter

```
setUtilisateur(precedent => ({ ...precedent, nom: nouveauNom }));
setDemandes(precedentes => [nouvelleDemande, ...precedentes]);
setDemandes(precedentes => precedentes.filter(d => d.id !== id));
```

L’état doit être remplacé par une nouvelle valeur. Modifier directement l’objet existant empêche React de détecter correctement le changement et peut produire des affichages obsolètes.

2.2 useReducer - état avec plusieurs actions

`useReducer` centralise les changements d’un état dans une fonction appelée reducer. Il renvoie l’état actuel et une fonction `dispatch` pour envoyer une action.

Quand l’utiliser : Quand plusieurs actions modifient ensemble un état lié, par exemple un formulaire complexe ou une liste avec ajouter, modifier, supprimer et réinitialiser.

Forme générale : `const [etat, dispatch] = useReducer(reducer, etatInitial)`

```
import { useReducer } from "react";

function reducer(etat, action) {
  switch (action.type) {
    case "ajouter":
      return [...etat, action.demande];
    case "supprimer":
      return etat.filter(d => d.id !== action.id);
    default:
      return etat;
  }
}

function Liste() {
  const [demandes, dispatch] = useReducer(reducer, []);
  // dispatch({ type: "supprimer", id: 12 });
}
```

À retenir Le reducer doit être pur : même état + même action = même nouvel état, sans appel réseau ni modification directe des données.

3. Partager une valeur avec useContext

3.1 useContext - lire un contexte

`useContext` lit la valeur fournie par le fournisseur le plus proche au-dessus du composant. React réaffiche le consommateur quand cette valeur change.

Quand l'utiliser : Pour des données nécessaires à plusieurs niveaux de l'application : utilisateur connecté, thème, langue ou permissions globales.

Forme générale : `const valeur = useContext(MonContexte)`

```
import { createContext, useContext } from "react";

const AuthContext = createContext(null);

function BoutonCompte() {
  const auth = useContext(AuthContext);
  return <span>{auth?.utilisateur?.email ?? "Invité"}</span>;
}

function App() {
  return (
    <AuthContext value={{ utilisateur }}>
      <BoutonCompte />
    </AuthContext>
  );
}
```

> À retenir Le fournisseur doit être au-dessus du composant qui appelle `useContext`. Pour un petit état local, les props ou `useState` sont souvent plus simples.

Dans GAIN OFFICE, un Hook tel que `useAuth()` peut lui-même appeler `useContext(AuthContext)` et fournir une API plus claire : utilisateur, connexion, déconnexion et vérification de permissions.

4. Hooks de références

4.1 useRef - garder une valeur sans nouveau rendu

`useRef` renvoie un objet stable contenant `current`. Changer cette propriété ne provoque pas de rendu.

Quand l'utiliser : Pour pointer vers un élément DOM, conserver l'identifiant d'un minuteur ou mémoriser une valeur technique qui ne doit pas être affichée.

Forme générale : `const reference = useRef(valeurInitiale)`

```
import { useRef } from "react";

function ChampRecherche() {
  const champRef = useRef(null);

  function donnerLeFocus() {
    champRef.current?.focus();
  }

  return (
    <>
      <input ref={champRef} />
      <button onClick={donnerLeFocus}>Rechercher</button>
    </>
  );
}
```

> À retenir Si une modification doit changer ce qui est visible, utilise plutôt `useState`. Une référence est une valeur persistante, pas un état d'affichage.

4.2 useImperativeHandle - limiter l'API d'une référence

`useImperativeHandle` personnalise les méthodes qu'un composant enfant expose à une référence reçue de son parent.

Quand l'utiliser : Rarement : seulement quand un parent doit déclencher une action impérative limitée, comme demander le focus à un composant champ.

Forme générale : `useImperativeHandle(ref, createHandle, dependencies)`

```
import { useImperativeHandle, useRef } from "react";

function Champ({ ref }) {
  const inputRef = useRef(null);
  useImperativeHandle(ref, () => ({
    focus() { inputRef.current?.focus(); }
  }), []);
  return <input ref={inputRef} />;
}
```

> À retenir Dans React 19, un composant peut recevoir `ref` comme prop. Les composants existants des versions antérieures utilisent souvent `forwardRef`. Préfère les props et les événements déclaratifs quand ils suffisent.

5. Synchroniser avec l'extérieur : les Effects

Un Effect sert à synchroniser React avec un système extérieur : API, abonnement, minuteur, lecteur vidéo ou bibliothèque non React. Il ne sert pas à recalculer des valeurs dérivées ou à orchestrer toute la logique d'un écran.

5.1 useEffect - effet après l'affichage

`useEffect` exécute une fonction après l'affichage. Elle peut renvoyer un nettoyage, que React exécute avant de relancer l'effet et au démontage.

Quand l'utiliser : Pour connecter un abonnement, un minuteur, ou synchroniser une valeur avec une API ou une API navigateur. Pour charger des données, un effet reste possible; les frameworks ou bibliothèques de requête peuvent aussi le gérer.

Forme générale : `useEffect(setup, dependencies)`

```
import { useEffect } from "react";

useEffect(() => {
  let actif = true;

  api.get("/demandes").then(response => {
    if (actif) setDemandes(response.data.demandes);
  }).catch(error => {
    if (actif) setErreur(error.message);
  });

  return () => { actif = false; };
}, []);
```

> À retenir Les dépendances incluent les valeurs réactives lues par l'effet. Tableau absent : après chaque rendu. `[]` : au montage et nettoyage au démontage. `[id]` : au montage puis quand `id` change. En Strict Mode, le développement peut effectuer un cycle supplémentaire de nettoyage et de configuration.

Toujours prévoir le nettoyage nécessaire

```
useEffect(() => {
  const timer = setInterval(actualiser, 5000);
  return () => clearInterval(timer);
}, [actualiser]);
```

Un bon nettoyage annule ou défait exactement ce que l'effet a installé. Pour une requête, empêcher une ancienne réponse d'écraser un écran plus récent; pour un abonnement, se désabonner; pour un minuteur, l'arrêter.

5.2 useLayoutEffect - mesurer avant le repeint

Même idée que `useEffect`, mais l'effet s'exécute après les mutations DOM de React et avant le repeint du navigateur.

Quand l'utiliser : Mesurer un élément et ajuster immédiatement son positionnement pour éviter un flash visuel. C'est un outil spécialisé.

Forme générale : `useLayoutEffect(setup, dependencies)`

```
import { useLayoutEffect, useRef, useState } from "react";

function Infobulle() {
  const ref = useRef(null);
  const [hauteur, setHauteur] = useState(0);
  useLayoutEffect(() => {
    setHauteur(ref.current.getBoundingClientRect().height);
  }, []);
  return <div ref={ref} data-hauteur={hauteur}>Info</div>;
}
```

> À retenir Il bloque le repeint et peut ralentir la page. Utilise `useEffect` sauf si l'affichage clignote ou si la mesure doit impérativement précéder le repeint.

5.3 useInsertionEffect - insertion de styles

Un effet très précoce, exécuté avant les modifications DOM de React. Il est prévu principalement pour les auteurs de bibliothèques qui injectent des styles dynamiques.

Quand l'utiliser : Presque jamais dans le code d'une application métier. Pour un composant React ordinaire, choisir CSS classique ou `useEffect`.

Forme générale : `useInsertionEffect(setup, dependencies)`

```
// Usage spécialisé dans une bibliothèque de styles :
useInsertionEffect(() => {
  injecterLaRegleCSS(regle);
}, [regle]);
```

> À retenir Ne pas y lire des références DOM pour faire des mesures et ne pas l'utiliser à la place de `useEffect`.

6. useEffectEvent - logique non réactive liée à un effet

6.1 useEffectEvent

Crée une fonction appelée depuis un effet qui lit les dernières valeurs des props et de l'état sans provoquer le redémarrage de l'effet pour ces valeurs.

Quand l'utiliser : Quand une connexion doit changer selon une dépendance précise, mais qu'un callback de cette connexion doit lire une valeur récente qui ne doit pas recréer la connexion.

Forme générale : `const evenement = useEffectEvent(callback)`

```
import { useEffect, useEffectEvent } from "react";

function Salon({ salonId, theme }) {
  const afficherConnexion = useEffectEvent(() => {
    notifier(`Connecté, thème ${theme}`);
  });

  useEffect(() => {
    const connexion = connecter(salonId);
    connexion.on("connect", afficherConnexion);
    connexion.open();
    return () => connexion.close();
  }, [salonId]);
}
```

À retenir Un Effect Event ne remplace pas les dépendances de l'effet. Ne l'appelle pas depuis un bouton ou un gestionnaire d'événement ordinaire; il est destiné à la logique déclenchée par un effet.

Pour débuter Tu peux apprendre React sans utiliser `useEffectEvent`. Comprends d'abord `useEffect`, ses dépendances et son nettoyage.

7. Rendu et performance

7.1 useMemo - mémoriser un calcul

Met en cache le résultat d'une fonction entre les rendus tant que ses dépendances ne changent pas.

Quand l'utiliser : Pour un calcul réellement coûteux ou une liste filtrée volumineuse. Une valeur calculable simplement peut rester une variable ordinaire.

Forme générale : `const resultat = useMemo(() => calcul(), [dependances])`

```
const demandesFiltrees = useMemo(() => {
  const terme = recherche.toLowerCase();
  return demandes.filter(d =>
    d.objet.toLowerCase().includes(terme)
  );
}, [demandes, recherche]);
```

> À retenir Ce n'est pas un outil pour rendre le code correct. Tu peux d'abord écrire le calcul directement, mesurer les performances, puis mémoriser si cela aide.

7.2 useCallback - mémoriser une fonction

Garde la même identité de fonction entre les rendus tant que ses dépendances ne changent pas.

Quand l'utiliser : Quand une fonction est transmise à un composant optimisé avec `memo`, ou quand une dépendance d'effet doit rester stable.

Forme générale : `const fonctionStable = useCallback(fonction, [dependances])`

```
const ouvrirDemande = useCallback((id) => {
  setSelection(id);
}, []);
```

> À retenir Créer une fonction dans un composant n'est pas automatiquement un problème. N'ajoute pas `useCallback` partout sans besoin mesuré.

7.3 useTransition - mettre à jour sans bloquer l'interface

Marque une mise à jour d'état comme transition non urgente. Le Hook renvoie aussi `isPending`.

Quand l'utiliser : Pour un filtrage ou un changement d'écran coûteux qui ne doit pas bloquer la saisie immédiate.

Forme générale : `const [isPending, startTransition] = useTransition()`

```
const [recherche, setRecherche] = useState("");
const [filtre, setFiltre] = useState("tous");
const [isPending, startTransition] = useTransition();

function choisirFiltre(valeur) {
  startTransition(() => setFiltre(valeur));
}

<input value={recherche} onChange={e => setRecherche(e.target.value)} />
{isPending && <span>Actualisation...</span>}
```

> À retenir Ne marque pas la mise à jour d'un champ contrôlé de saisie comme transition. L'entrée doit rester immédiate.

7.4 useDeferredValue - retarder une valeur non urgente

Renvoie une version différée d'une valeur. L'interface peut afficher la nouvelle saisie immédiatement tout en laissant une liste lourde se mettre à jour ensuite.

Quand l'utiliser : Quand une valeur change souvent, comme une recherche, et qu'un rendu dépendant est coûteux.

Forme générale : `const valeurDifferee = useDeferredValue(valeur)`

```
const [recherche, setRecherche] = useState("");
const rechercheDifferee = useDeferredValue(recherche);
const resultats = filtrerEnregistrements(liste, rechercheDifferee);
```

> À retenir Ce Hook ne réduit pas le nombre de requêtes réseau et ne remplace pas un debounce. Il joue sur la priorité de rendu de React.

8. Identifiants et systèmes externes

8.1 useId - identifiants d'accessibilité

Génère un identifiant stable qui convient pour relier un label, un champ et un message d'aide.

Quand l'utiliser : Pour des attributs tels que `htmlFor`, `aria-describedby` ou `aria-labelledby`.

Forme générale : `const id = useId()`

```
import { useId } from "react";

function ChampEmail() {
  const id = useId();
  return (
    <>
      <label htmlFor={id}>Adresse e-mail</label>
      <input id={id} type="email" aria-describedby={`${id}-aide`} />
      <small id={`${id}-aide`}>Utilisez votre adresse professionnelle.</small>
    </>
  );
}
```

> À retenir Ne l'utilise pas comme clé de ligne pour `map()`. Les clés doivent venir des identifiants des données, par exemple `demande.id_demande`.

8.2 useSyncExternalStore - abonnement à un magasin externe

Permet de lire un magasin d'état qui vit en dehors de React et de s'abonner à ses changements.

Quand l'utiliser : Pour intégrer un magasin externe ou une API navigateur avec un modèle d'abonnement. Les états ordinaires doivent généralement rester dans React.

Forme générale : `const snapshot = useSyncExternalStore(subscribe, getSnapshot, getServerSnapshot?)`

```
const online = useSyncExternalStore(
  callback => {
    window.addEventListener("online", callback);
    window.addEventListener("offline", callback);
    return () => {
      window.removeEventListener("online", callback);
      window.removeEventListener("offline", callback);
    };
  },
  () => navigator.onLine
);
```

> À retenir La fonction d'abonnement doit renvoyer une fonction de désabonnement. Le snapshot doit rester stable tant que la donnée n'a pas changé.

8.3 useDebugValue - information dans React DevTools

Ajoute un libellé de diagnostic pour un Hook personnalisé dans React DevTools.

Quand l'utiliser : Pour aider à inspecter des Hooks personnalisés partagés dans une bibliothèque ou une application conséquente.

Forme générale : `useDebugValue(valeur, formatter?)`

```
function useConnexion(socket) {
  const statut = useStatutSocket(socket);
  useDebugValue(statut, s => `Connexion : ${s}`);
  return statut;
}
```

À retenir Ce Hook ne modifie pas l’interface et n’est presque jamais nécessaire dans les composants ordinaires.

9. Actions, formulaires et mises à jour optimistes

9.1 useActionState - résultat et attente d'une action

Associe une action à un état de résultat et fournit un indicateur de traitement. L'action reçoit l'état précédent et les données soumises.

Quand l'utiliser : Pour les formulaires dont la soumission produit un résultat, un message d'erreur ou un état de progression, notamment avec les Actions React.

Forme générale : `const [state, action, isPending] = useActionState(fn, initialState)`

```
import { useActionState } from "react";

async function enregistrer(etatPrecedent, formData) {
  const objet = formData.get("objet");
  const resultat = await api.post("/demandes", { objet });
  return { message: resultat.data.message };
}

function FormulaireDemande() {
  const [state, formAction, pending] = useActionState(enregistrer, { message: "" });
  return (
    <form action={formAction}>
      <input name="objet" required />
      <button disabled={pending}>Envoyer</button>
      <p>{state.message}</p>
    </form>
  );
}
```

> À retenir Pour les formulaires classiques, `useState` et un gestionnaire `onSubmit` restent une solution valide. `useActionState` est particulièrement utile avec le modèle Action de React.

9.2 useOptimistic - afficher une réponse immédiate

Crée une version provisoire de l'état pendant qu'une action asynchrone se termine. Si l'action échoue, la donnée de base reste la source de vérité.

Quand l'utiliser : Pour une action dont le résultat paraît évident et doit sembler instantané : marquer une notification comme lue, aimer un élément ou afficher une nouvelle ligne en attente.

Forme générale : `const [optimistic, setOptimistic] = useOptimistic(base, reducer?)`

```
import { useOptimistic, useState, useTransition } from "react";

function BoutonLu({ initialementLu, enregistrer }) {
  const [lu, setLu] = useState(initialementLu);
  const [optimiste, setOptimiste] = useOptimistic(lu);
  const [, startTransition] = useTransition();

  function marquerLu() {
    startTransition(async () => {
      setOptimiste(true);
      const resultat = await enregistrer();
      setLu(resultat.lu);
    });
  }
  return <button onClick={marquerLu}>{optimiste ? "Lue" : "Non lue"}</button>;
}
```

> À retenir Le setter optimiste s'utilise pendant une Action ou une transition. L'état réel doit quand même être mis à jour lorsque le serveur confirme le changement.

9.3 useFormStatus - état du formulaire parent

Hook de `react-dom` qui lit le statut de soumission du formulaire parent le plus proche : `pending`, les données et la méthode.

Quand l'utiliser : Pour un bouton de soumission réutilisable qui affiche ou désactive le bouton pendant l'envoi.

Forme générale : `const { pending, data, method, action } = useFormStatus()`

```
import { useFormStatus } from "react-dom";

function BoutonEnvoyer() {
  const { pending } = useFormStatus();
  return <button disabled={pending}>{pending ? "Envoi..." : "Envoyer"}</button>;
}

function Formulaire() {
  return <form action={envoyer}><input name="objet" /><BoutonEnvoyer /></form>;
}
```

> À retenir Place `BoutonEnvoyer` comme enfant du formulaire. Appeler `useFormStatus` dans le composant qui crée lui-même le formulaire ne lit pas le formulaire retourné par ce même composant.

10. Créer un Hook personnalisé

Un Hook personnalisé est une fonction JavaScript ordinaire qui utilise d'autres Hooks et dont le nom commence par `use`. Il partage une logique, pas une instance d'état : chaque composant qui appelle le Hook garde ses propres états, sauf si le Hook s'appuie sur un contexte ou un magasin partagé.

Exemple : récupérer les demandes

```
import { useEffect, useState } from "react";
import api from "../config/apiClient";

export function useDemandes() {
  const [demandes, setDemandes] = useState([]);
  const [chargement, setChargement] = useState(true);
  const [erreur, setErreur] = useState("");

  useEffect(() => {
    let actif = true;
    api.get("/demandes")
      .then(reponse => {
        if (actif) setDemandes(reponse.data.demandes ?? []);
      })
      .catch(e => {
        if (actif) setErreur(e.message);
      })
      .finally(() => {
        if (actif) setChargement(false);
      });
    return () => { actif = false; };
  }, []);

  return { demandes, chargement, erreur };
}
```

La page peut ensuite faire `const { demandes, chargement, erreur } = useDemandes()`. Le chargement est ainsi séparé de l'affichage de la page.

> Nommer correctement Le préfixe `use` permet aux outils React et au lint de repérer les règles des Hooks. N'appelle pas un Hook depuis une fonction utilitaire ordinaire qui ne s'exécute pas pendant le rendu.

11. Quel Hook choisir ?

Besoin	Choix courant	À éviter
Une valeur affichée change au clic	useState	Modifier une variable locale
Plusieurs actions modifient un état complexe	useReducer	Éparpiller des mises à jour liées
Une donnée est partagée à plusieurs niveaux	useContext	Chaîner beaucoup de props sans raison
Charger une API / gérer un abonnement	useEffect ou une bibliothèque dédiée	Utiliser un effet pour recalculer une valeur
Un calcul lent dépend de données	useMemo après mesure	Mémoriser chaque expression
Une valeur technique ne doit pas afficher à nouveau	useRef	Mettre toute donnée dans une ref
Relier label et champ	useId	Employer cet ID pour une clé de liste
État global existant hors React	useSyncExternalStore	Dupliquer son état localement
Action de formulaire avec réponse / attente	useActionState	Gérer plusieurs états incohérents
Réponse immédiate en attente du serveur	useOptimistic	Confondre résultat provisoire et résultat confirmé

Dans la page Suivi des demandes

- `useState` garde les demandes, la recherche, les filtres et la demande sélectionnée.
- `useEffect` synchronise la page avec l'API Laravel au chargement.
- `useMemo` évite de recalculer les colonnes et les demandes filtrées à chaque rendu.
- `react-data-table-component` gère ensuite l'affichage, le tri et la pagination; ce n'est pas un Hook.

> Distinction utile Un Hook donne une capacité React. Une bibliothèque comme `react-data-table-component` fournit un composant prêt à l'emploi. Les deux peuvent être utilisés ensemble.

12. Erreurs fréquentes et bonnes pratiques

1. Hook appelé conditionnellement

```
// Incorrect
if (connecte) {
  const [menu, setMenu] = useState(false);
}

// Correct : appeler le Hook avant le conditionnel
const [menu, setMenu] = useState(false);
if (!connecte) return <Connexion />;
```

2. Dépendances d'effet manquantes

Si l'effet lit `utilisateurId`, cette valeur doit figurer dans ses dépendances. Sinon l'effet peut utiliser une ancienne valeur. Ne supprime pas une dépendance uniquement pour forcer l'effet à ne tourner qu'une fois : reformule l'effet ou déplace la logique.

3. L'effet sert à calculer une valeur

```
// À éviter : état dérivé dupliqué et effet
useEffect(() => {
  setFiltrees(demandes.filter(...));
}, [demandes]);

// Préférable : valeur dérivée
const filtrees = demandes.filter(...);
```

4. Mutation directe de l'état

```
// Incorrect
demandes.push(nouvelleDemande);
setDemandes(demandes);

// Correct
setDemandes(precedentes => [...precedentes, nouvelleDemande]);
```

5. Optimisation prématurée

Un rendu React normal est attendu. Commence par un code clair, puis profile l'application. Ajoute `useMemo` ou `useCallback` quand ils évitent réellement un travail coûteux ou un rendu inutile.

6. Oublier le nettoyage

Tout effet qui ouvre un abonnement, ajoute un écouteur, démarre un minuteur ou crée une connexion doit généralement rendre une fonction de nettoyage.

> Règle pratique Pour chaque `useEffect`, demande-toi : quel système extérieur est synchronisé ? Quelle action inverse le branchement ? Si aucune réponse n'existe, l'effet est peut-être inutile.

13. Fiche de référence des Hooks

Hook	Usage en une phrase	Fréquence
useState	État local qui déclenche un rendu	Quotidien
useReducer	État complexe avec actions	Régulier
useContext	Valeur partagée dans l'arbre	Régulier
useRef	Référence DOM ou valeur non visuelle	Régulier
useEffect	Synchronisation avec l'extérieur	Régulier, avec soin
useMemo	Résultat de calcul mémorisé	Quand utile
useCallback	Fonction mémorisée	Quand utile
useId	ID accessible stable	Formulaires
useTransition	Mise à jour non urgente	Avancé
useDeferredValue	Valeur différée	Avancé
useLayoutEffect	Mesure avant repeint	Rare
useInsertionEffect	Injection CSS précoce	Bibliothèques
useEffectEvent	Callback d'effet avec valeurs récentes	Avancé
useImperativeHandle	API impérative exposée au parent	Rare
useSyncExternalStore	Abonnement à un état extérieur	Intégration
useDebugValue	Diagnostic DevTools d'un Hook	Bibliothèques
useActionState	État de résultat d'une Action	Formulaires React 19
useOptimistic	État temporaire pendant une Action	Réponse immédiate
useFormStatus	Statut du formulaire parent	React DOM

Les six à apprendre en premier

useState, useEffect, useContext, useRef, useMemo et useCallback. Ajoute useReducer quand l'état devient plus difficile à organiser. Les autres répondent à des besoins plus précis.

14. Glossaire

Terme	Sens
Rendu	Appel du composant par React pour déterminer le JSX à afficher.
État	Valeur conservée par React entre les rendus.
Dépendance	Valeur dont le changement peut relancer un effet ou invalider un cache.
Effet	Code qui synchronise un composant avec un système extérieur.
Nettoyage	Fonction qui annule un abonnement ou une ressource créée par un effet.
Hook personnalisé	Fonction commençant par use qui réutilise des Hooks.
Action	Fonction qui met à jour des données, souvent de façon asynchrone, dans le modèle React 19.
État optimiste	Affichage provisoire avant confirmation du serveur.

Documentation officielle

Les signatures et recommandations évoluent avec React. Pour un comportement précis, consulter la référence officielle de la version utilisée par le projet.

Hooks intégrés React - https://react.dev/reference/react/hooks

Règles des Hooks - https://react.dev/reference/rules/rules-of-hooks

useState - https://react.dev/reference/react/useState

useEffect - https://react.dev/reference/react/useEffect

useOptimistic - https://react.dev/reference/react/useOptimistic

API use - https://react.dev/reference/react/use

Hooks React DOM - https://react.dev/reference/react-dom/hooks

useFormStatus - https://react.dev/reference/react-dom/hooks/useFormStatus

> À retenir pour GAIN OFFICE Commence par `useState` pour les interactions, `useEffect` pour les appels et abonnements externes, puis `useMemo` seulement si le calcul des filtres devient coûteux. Garde les règles métier et les permissions vérifiées aussi côté Laravel.